

# Architecture Overview

## Goal

The system is designed to collect structured intelligence from public web sources, validate freshness, normalize entities, and export records in a spreadsheet-friendly format. The final task requires a public Google Sheet and a GitHub repo containing the implementation and design artifacts.

## Core pipeline

```
source registry
-> async fetch workers
-> parse RSS/HTML/XML
-> freshness validation
-> schema normalization
-> entity resolution
-> export to JSONL/CSV
-> manual Google Sheet upload
```

The current repository implements the local pipeline foundation. It is not yet a complete 500k-scale production deployment, but it demonstrates the right architecture and safe engineering patterns.

## Scale strategy

A production version would use:

- a queue system such as Redis or Kafka
- multiple worker pools by source type
- bounded concurrency per domain
- idempotent deduplication keyed by canonical URL and content hash
- per-source cursor tracking
- object storage for raw HTML and feed snapshots

This enables horizontal scaling without changing the core logic. The repo's current implementation keeps the same design pattern while using a simpler local prototype.

## Freshness strategy

The solution requires all news and job items to be from the last 24 hours. The implementation enforces this by:

- parsing ISO and RSS dates
- handling relative dates like "2 hours ago"

- rejecting missing or stale timestamps
- keeping source URL as provenance for each record

This provides a reliable freshness gate before record acceptance.

## LLM handling

Large language model extraction is handled through a provider fallback pattern. The design is intended to protect against:

- 429 Too Many Requests
- 413 Payload Too Large
- provider-specific failures

The implementation includes:

- retry with jitter
- payload chunking by paragraph boundary
- provider fallback chain

## Entity resolution

The resolver normalizes names and compares them against canonical labels. This is the foundation for mapping messy names like:

- OpenAI
- Open AI
- OpenAI, Inc.

The architecture stores mapping events so each raw entity can be traced back to its canonical form.

## Storage strategy

The current local pipeline writes:

- JSONL as the raw immutable export
- CSV tabs as spreadsheet-friendly output

For a larger deployment, the production design would add:

- PostgreSQL for canonical records and source metadata
- object storage for raw HTML/text
- pgvector for semantic matching
- graph storage or relational edges for startup-product-founder-paper relationships

## Important submission note

The Google Sheet is a required manual submission artifact. The codebase provides the export and tab structure, but the final public sheet itself must be created in Google Drive and made publicly accessible.

This architecture intentionally separates:

- the engineering pipeline in code
- the final public reporting layer in Google Sheets

That separation is practical and mirrors the assessment requirements.